

Лекция 2. Элементы базового интерфейса

- Создание и управление средой разработки.
- Прimitives графического интерфейса
- Деревья представлений
- XML-элементы и атрибуты.
- Инициализация представлений.
- Стандартные разметки.

Для работы с Android необходимо настроить среду разработки. Это включает установку инструментов для написания кода, его компиляции и отладки. Современные разработчики используют Android Studio — официальную IDE от Google, которая включает встроенные SDK, эмулятор и инструменты для работы с XML. Java Development Kit (JDK) — набор инструментов для компиляции Java-кода. Даже если вы пишете на Kotlin, JDK требуется для работы Android Studio.

- Скачайте JDK 11+ с [официального сайта Oracle](https://www.oracle.com/technetwork/java/javase-downloads-1344955.html).
- Установите JDK и настройте переменную окружения JAVA_HOME, указав путь к папке (например, C:\Program Files\Java\jdk-17).

Прimitives графического интерфейса — это базовые элементы, из которых строится визуальная часть Android-приложения. Они позволяют пользователю взаимодействовать с программой: вводить данные, нажимать кнопки, выбирать опции.

Рассмотрим четыре ключевых элемента: Button, EditText, ProgressBar и CheckBox, а также их настройку через XML и код.

Элемент Button (Кнопка).

Назначение: Выполнение действий по нажатию (например, отправка формы, переход между экранами).

Основные атрибуты в XML:

```
<Button
    android:id="@+id/btnSubmit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Отправить"
    android:textColor="#FFFFFF"
    android:background="#2196F3"
    android:padding="16dp"
    android:enabled="true" />
```

- android:id — уникальный идентификатор для доступа к кнопке из кода.

Пример: btnSubmit позволяет манипулировать кнопкой в Kotlin.

- android:text — текст, отображаемый на кнопке.

Совет: Используйте ресурсы строк (@string/submit) для мультиязычности.

- android:background — цвет или изображение фона.

Пример: Используйте ?attr/selectableItemBackground для эффекта клика.

- android:enabled — активность кнопки.

Практика: Установите false, чтобы заблокировать кнопку до заполнения формы.

- android:padding — отступы между текстом и границами кнопки.

- Совет: Используйте android:paddingStart и android:paddingEnd для RTL-поддержки.

Обработка клика:

```
val btnSubmit: Button = findViewById(R.id.btnSubmit)
btnSubmit.setOnClickListener {
    // Действие при нажатии
    Toast.makeText(this, "Форма отправлена", Toast.LENGTH_SHORT).show()
}
```

Пример с переходом на другую активность:

```
val intent = Intent(this, SuccessActivity::class.java)
startActivity(intent)
```

Использование анонимного класса:

```
btnSubmit.setOnLongClickListener(object : View.OnLongClickListener {
```

```

override fun onLongClick(v: View?): Boolean {
    Toast.makeText(this@MainActivity, "Долгое нажатие",
        Toast.LENGTH_SHORT).show()
    return true
}
})

```

Элемент **EditText** (Поле ввода)

Назначение: Ввод текста (логины, пароли, сообщения).

Основные атрибуты в XML:

```

<EditText
    android:id="@+id/etEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Введите email"
    android:inputType="textEmailAddress"
    android:maxLines="1"
    android:textSize="16sp" />

```

- `android:hint` — подсказка, исчезающая при вводе.

Совет: Укажите пример формата ввода, например, "example@mail.com".

- `android:inputType` — тип вводимых данных.

Примеры:

`textPassword` — скрывает символы.

`number` — клавиатура для чисел.

`textCapWords` — автоматическое заглавные буквы.

- `android:imeOptions` — управление клавиатурой.

Пример: `actionDone` для завершения ввода:

```
android:imeOptions="actionDone"
```

Получение текста из **EditText**:

```

val etEmail: EditText = findViewById(R.id.etEmail)
val email = etEmail.text.toString()

```

```
if (email.isEmpty()) {
    etEmail.error = "Поле не может быть пустым"
}
```

Пример валидации email:

```
val emailRegex = "[a-zA-Z0-9._-]+@[a-z]+\\.+[a-z]+".toRegex()
if (!email.matches(emailRegex)) {
    etEmail.error = "Некорректный email"
}
```

TextWatcher для отслеживания ввода:

```
etEmail.addTextChangedListener(object : TextWatcher {
    override fun afterTextChanged(s: Editable?) {
        // Обработка изменений
    }
    override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
    override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
})
```

Элемент ProgressBar (Индикатор прогресса)

Назначение: Визуализация процесса (загрузка данных, выполнение задачи).

Основные атрибуты в XML:

- `android:id`

Уникальный идентификатор для доступа к элементу из кода.

Пример: `android:id="@+id/progressBar"`

- `android:layout_width / android:layout_height`

Размеры элемента. Значения: `wrap_content`, `match_parent`, конкретные размеры в `dp`.

Пример: `android:layout_width="200dp"`

- `android:visibility`

Видимость элемента. Значения: `visible`, `invisible`, `gone`.

Пример: `android:visibility="gone"`

- `style`
Задаёт стиль `ProgressBar`. Для горизонтального индикатора:
`style="?android:attr/progressBarStyleHorizontal"`
Для
кругового: `style="@style/Widget.AppCompat.ProgressBar"`.
- `android:indeterminate`
Определяет тип индикатора:
 - `true` — бесконечная анимация (`Indeterminate`).
 - `false` — детерминированный индикатор (`Determinate`)
- `android:max`
Максимальное значение прогресса (по умолчанию 100).
Пример: `android:max="100"`
- `android:progress`
Текущее значение прогресса (от 0 до `max`).
Пример: `android:progress="50"`
- `android:secondaryProgress`
Вторичный прогресс (например, для отображения буферизации).
Пример: `android:secondaryProgress="75"`
- `android:progressTint`
Цвет заполненной части индикатора.
Пример: `android:progressTint="@color/blue"`
- `android:progressBackgroundTint`
Цвет фона индикатора.
Пример: `android:progressBackgroundTint="@color/gray"`
"

- `android:minWidth` / `android:minHeight`

Минимальные размеры элемента.

Пример: `android:minWidth="50dp"`

Типы **ProgressBar**

Круговой (по умолчанию):

```
<ProgressBar
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

Горизонтальный:

```
<ProgressBar
    style="@android:style/Widget.ProgressBar.Horizontal"
    android:layout_width="match_parent"
    android:layout_height="8dp"
    android:max="100"
    android:progress="30" />
```

Совет: Используйте `MaterialProgressBar` из библиотеки `Material Components` для современного дизайна.

Управление прогрессом из кода:

```
val progressBar: ProgressBar = findViewById(R.id.progressBar)
progressBar.progress = 75 // Установка значения (от 0 до max)
```

Пример динамического изменения:

```
val handler = Handler(Looper.getMainLooper())
Thread {
    for (i in 0..100) {
        Thread.sleep(50)
        handler.post { progressBar.progress = i }
    }
}.start()
```

Элемент **CheckBox** (Флажок)

Назначение: Выбор одной или нескольких опций (например, согласие с условиями).

Основные атрибуты в XML:

```
<CheckBox
    android:id="@+id/cbTerms"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Принимаю условия соглашения"
    android:checked="false"
    android:buttonTint="#4CAF50" />
```

- android:checked — начальное состояние (true — выбрано).

Совет: Установите false по умолчанию, если пользователь должен явно согласиться.

- android:buttonTint — цвет флажка.

Пример: Используйте цвета из вашей палитры Material.

Обработка изменения состояния:

```
val cbTerms: CheckBox = findViewById(R.id.cbTerms)
cbTerms.setOnCheckedChangeListener { _, isChecked ->
    if (isChecked) {
        // Флажок отмечен
        btnSubmit.isEnabled = true
    } else {
        // Флажок снят
        btnSubmit.isEnabled = false
    }
}
```

Совместное использование элементов

Пример формы регистрации:

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/etUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Имя пользователя" />
```

```
<EditText
    android:id="@+id/etPassword"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Пароль"
    android:inputType="textPassword" />

<CheckBox
    android:id="@+id/cbRemember"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Запомнить меня" />

<Button
    android:id="@+id/btnLogin"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Войти" />

</LinearLayout>
```

Деревья представлений

Интерфейс Android-приложения реализуется как иерархическая структура, состоящая из объектов класса View и его подклассов. Корневым элементом иерархии является контейнер (например, ConstraintLayout), который управляет расположением и отрисовкой дочерних элементов. Эта структура называется деревом представлений.

View (Представление) – это базовый класс для всех элементов интерфейса (TextView, Button, ImageView и т.д.), отвечает за отрисовку на экране и обработку событий взаимодействия с пользователем. ViewGroup является абстрактным классом, который служит родительским классом для всех контейнеров в Android. Он предоставляет базовую функциональность для:

- Хранения дочерних элементов (объектов View).
- Управления их расположением и отрисовкой.
- Обработки событий взаимодействия с пользователем.

В Android существует множество классов, которые наследуются от ViewGroup , предоставляя разные стратегии управления дочерними элементами. Рассмотрим основные из них.

ConstraintLayout

Назначение: Современный гибкий контейнер для создания сложных макетов с точным позиционированием элементов.

Особенности:

- Позволяет позиционировать элементы относительно друг друга и границ экрана.
- Использует ограничения (constraints) вместо вложенных контейнеров.
- Поддерживает анимацию и жесты через MotionLayout.

Пример:

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/btn1"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent" />

    <Button
        android:id="@+id/btn2"
        app:layout_constraintTop_toBottomOf="@id/btn1"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

LinearLayout

Назначение: Располагает элементы вертикально или горизонтально.

Особенности:

- Простой и интуитивно понятный.
- Может использовать веса (weight) для распределения пространства.

Пример:

```
<LinearLayout
  android:orientation="vertical">
  <Button android:text="1" />
  <Button android:text="2" />
</LinearLayout>
```

FrameLayout

Назначение: Простейший контейнер для наложения элементов друг на друга.

Особенности:

- Все дочерние элементы прикрепляются к левому верхнему углу.
- Используется для слоёв (например, загрузочных экранов).

Пример:

```
<FrameLayout>
  <ImageView android:src="@drawable/background" />
  <TextView android:text="Центральный текст" />
</FrameLayout>
```

GridLayout

Назначение: Располагает элементы в виде сетки (строки и столбцы).

Особенности:

- Упрощает создание таблиц.
- Устаревает в пользу ConstraintLayout.

```
<GridLayout
  android:columnCount="2"
  android:rowCount="2">

  <Button android:text="1" />
  <Button android:text="2" />
  <Button android:text="3" />
  <Button android:text="4" />
</GridLayout>
```

TableLayout

Назначение: Создание таблиц с помощью TableRow.

Особенности:

- Каждый TableRow — строка таблицы.
- Колонки автоматически выравниваются.

```
<TableLayout>
  <TableRow>
    <TextView android:text="Имя" />
    <TextView android:text="Возраст" />
  </TableRow>

  <TableRow>
    <TextView android:text="Анна" />
    <TextView android:text="25" />
  </TableRow>
</TableLayout>
```

CoordinatorLayout

Назначение: Расширенный контейнер для сложных взаимодействий между элементами.

Особенности:

- Поддерживает поведение элементов (например, скрытие AppBar при прокрутке).
- Чаще всего используется с AppBarLayout, FloatingActionButton.

Пример:

```
<androidx.coordinatorlayout.widget.CoordinatorLayout>
  <com.google.android.material.appbar.AppBarLayout>
    <androidx.appcompat.widget.Toolbar ... />
  </com.google.android.material.appbar.AppBarLayout>

  <androidx.core.widget.NestedScrollView>
    <!-- Контент -->
  </androidx.core.widget.NestedScrollView>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Инициализация представлений

Инициализация представлений — это процесс подключения элементов интерфейса (кнопок, текстовых полей, изображений и т.д.) из XML-разметки к коду Kotlin/Java, чтобы можно было управлять их поведением

и отображением. Без инициализации невозможно взаимодействовать с элементами на экране. Рассмотрим основные методы инициализации.

Метод findViewById (традиционный способ)

Описание: Поиск элемента по его ID, указанному в XML.

Пример:

```
val button: Button = findViewById(R.id.myButton)
button.setOnClickListener {
    Toast.makeText(this, "Кнопка нажата", Toast.LENGTH_SHORT).show()
}
```

XML-разметка:

```
<Button
    android:id="@+id/myButton"
    android:text="Нажми меня" />
```

View Binding (современный способ)

Описание: Автоматическая генерация класса с ссылками на все элементы XML-файла.

Включение в проект: в build.gradle.kts (модульный файл) добавьте:

```
android {
    buildFeatures {
        viewBinding = true
    }
}
```

Пример использования:

```
private lateinit var binding: ActivityMainBinding
override fun onCreate(savedInstanceState: Bundle?) {
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)
    binding.myButton.setOnClickListener {
        binding.textView.text = "Текст изменён!"
    }
}
```

XML-разметка:

```
<LinearLayout ...>
```

```
<TextView
    android:id="@+id/textView"
    android:text="Исходный текст" />
<Button
    android:id="@+id/myButton"
    android:text="Изменить текст" />
</LinearLayout>
```

Data Binding (для связывания данных с интерфейсом)

Описание: Позволяет напрямую связывать данные из кода с элементами XML.

Включение в проект: в `build.gradle.kts` добавьте:

```
android {
    buildFeatures {
        dataBinding = true
    }
}
```

Пример использования:

```
val binding: ActivityMainBinding = DataBindingUtil.setContentView(this,
R.layout.activity_main)
val user = User("Анна", 25)
binding.user = user
```

XML-разметка с привязкой данных:

```
<layout>
    <data>
        <variable name="user" type="com.example.User" />
    </data>
    <LinearLayout ...>
        <TextView
            android:text="@{user.name}" />
        <TextView
            android:text="@{String.valueOf(user.age)}" />
    </LinearLayout>
</layout>
```

Традиционный метод `findViewById` остается актуальным благодаря своей простоте: он позволяет находить элементы по их идентификатору, указанному в разметке, и работать с ними напрямую. Однако этот подход

имеет ограничения, особенно в крупных проектах, где ручное привязывание каждого элемента увеличивает риск ошибок, таких как опечатки в ID или неправильное приведение типов. Например, если указать несуществующий ID или случайно изменить его в XML, приложение может аварийно завершиться из-за `NullPointerException`.

Современный подход через View Binding решает эти проблемы, автоматически генерируя классы, которые содержат ссылки на все элементы макета. Это обеспечивает безопасность типов: ошибка компиляции возникнет сразу, если элемент отсутствует или указан неверно. Кроме того, View Binding упрощает доступ к элементам через интуитивный синтаксис `binding.elementId`, исключая необходимость повторять `findViewById` для каждого элемента. Его использование особенно оправдано в новых проектах, так как он не требует сложных настроек и сочетает производительность с читаемостью кода. Однако для работы с этим методом необходимо включить соответствующую опцию в Gradle, что может быть незнакомо новичкам.

Если задача включает привязку данных к интерфейсу, например, в рамках архитектуры MVVM, Data Binding становится мощным инструментом. Он позволяет связывать поля объектов напрямую с элементами XML, автоматически обновляя UI при изменении данных. Это снижает объем кода, необходимого для ручного обновления текста, изображений или других атрибутов. Однако сложность настройки и потенциальное влияние на производительность (из-за генерации дополнительного кода) делают его менее подходящим для простых проектов. Data Binding также требует внимательного обращения с выражениями в разметке, чтобы избежать логических ошибок или утечек памяти.

Выбор метода зависит от масштаба задачи и архитектуры приложения. Для большинства случаев View Binding считается оптимальным решением, обеспечивая баланс между простотой и надежностью. Data

Binding же оправдан, когда требуется тесная интеграция данных и интерфейса, например, при работе с LiveData или ViewModel.

Инициализация представлений в Activity обычно происходит в методе onCreate() сразу после создания активности:

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = ActivityMainBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
    // Доступ к элементам через binding  
}
```

В Fragment процесс аналогичен, но требует дополнительных шагов: создание nullable-переменной для хранения binding-класса в onCreateView() и его очистка в onDestroyView() для предотвращения утечек памяти.

```
private var _binding: FragmentMainBinding? = null  
private val binding get() = _binding!!  
  
override fun onCreateView(  
    inflater: LayoutInflater,  
    container: ViewGroup?,  
    savedInstanceState: Bundle?  
): View {  
    _binding = FragmentMainBinding.inflate(inflater, container, false)  
    return binding.root  
}  
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
    super.onViewCreated(view, savedInstanceState)  
    binding.button.setOnClickListener { ... }  
}  
override fun onDestroyView() {  
    super.onDestroyView()  
    _binding = null // Освобождение ресурсов  
}
```

В итоге, выбор метода зависит от контекста: для базовых задач подойдет findViewById, но для современных приложений предпочтительнее использовать View Binding или Data Binding, учитывая их преимущества в безопасности типов, читаемости кода и интеграции с архитектурными паттернами.